实验三报告

姓名：李祥熙　学号：2234412799

2026 年 4 月 22 日

1　实验目的

本实验围绕 SQuAD 阅读理解数据集，使用两种基于 Transformer 的问答建模方式完成自动问答任务，目标如下：

- 完成基于 Transformer Encoder 的答案区间预测（start/end）任务。
- 完成基于 Encoder-Decoder 的序列到序列问答生成任务。
- 在示例代码基础上调节关键训练参数（学习率、batch 大小、训练轮次、并行加载参数等），观察其对训练与测试结果的影响。

2　实验过程

2.1　实验环境

- 操作系统：Linux, MacOS
- 编程语言：Python 3
- 深度学习框架：PyTorch
- 关键依赖：transformers、tqdm
- 结果输出目录：`model/`、`result/`

2.2　数据集与预处理

- 数据集：SQuAD v2.0（`SQuAD-train-v2.0.json` 与 `SQuAD-dev-v2.0.json`）。
- 输入形式：`[CLS] question [SEP] context [SEP]`。
- 任务一标签：答案起止位置索引（start_pos, end_pos）。

- 任务二标签：答案文本序列 `[CLS] answer [SEP]`。
- 分词方式：BERT tokenizer（`bert-base-uncased`），最大长度 256 或 384。

2.3 实现步骤

1. 阅读并保留给出的即成 `original_transformerQA1.py` 与 `original_transformerQA2.py` 作为参考。
2. 在 `transformerQA1.py` 中完成速度与训练稳定性优化(AMP、DataLoader 并行、batch-first 编码器等)，并训练 3 个 epoch 保存模型。
3. 在 `transformerQA2.py` 中修正自定义 Encoder-Decoder 的掩码与张量维度流，完成训练与测试。
4. 对两个任务分别执行测试，记录指标并生成结果截图。

3 实验内容

3.1 任务一：Transformer Encoder 区间抽取问答

该任务把问答建模为答案起点和终点的分类问题。模型输出序列每个位置的起点/终点 logits：

$$\mathcal{L} = \mathrm{CE}(\hat{y}_{start}, y_{start}) + \mathrm{CE}(\hat{y}_{end}, y_{end}) \tag{1}$$

主要参数：

- 批量大小：8
- 学习率：3×10^{-5}
- 训练轮次：3
- 最大序列长度：384
- 模型主干：`nn.TransformerEncoder`（$d_{model} = 768$, $nhead = 12$, $num_layers = 7$）

相对基线代码的主要改动：

- 启用混合精度训练（`autocast + GradScaler`）减少显存占用并提升吞吐。
- 将编码器改为 `batch_first=True`，减少频繁转置开销。
- 增加 DataLoader 并行参数(`num_workers/pin_memory/prefetch_factor/persistent_workers`)。
- 增加 `non_blocking` 数据传输、`torch.inference_mode()` 与可选 `torch.compile`。

3.2 任务二：自定义 Encoder-Decoder 生成式问答

该任务将问题与上下文编码后，由解码器自回归生成答案序列，损失为词表级交叉熵：

$$\mathcal{L} = \sum_{t=1}^{T} \mathrm{CE}(\hat{y}_t, y_t) \tag{2}$$

主要参数：

- 批量大小：8
- 学习率：3×10^{-5}
- 训练轮次：3
- 最大序列长度：256
- 模型主干：自实现 `TransformerEncoder/TransformerDecoder`（来自 `transformerRaw.py`）

相对基线代码的主要改动：

- 统一张量布局为 batch-first，移除训练/评估阶段不必要的转置。
- 修正解码器调用时的掩码传递，显式传入 memory mask。
- 完善 `generate_answer` 的推理流程（embedding、位置编码、掩码、自回归终止条件）。

4 实验结果

4.1 任务一结果：三轮模型测试对比

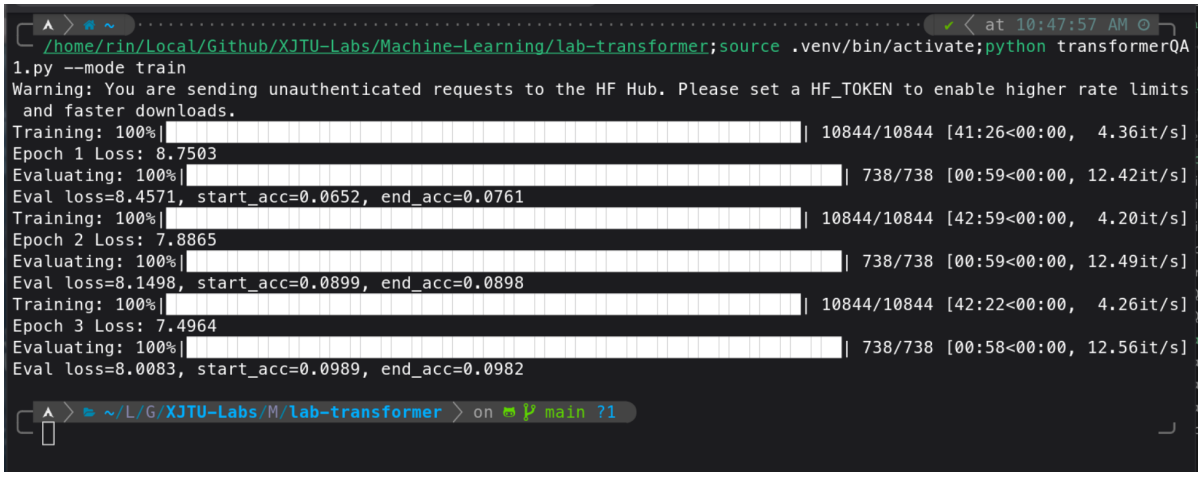

图 1: 任务一训练过程与评估日志截图

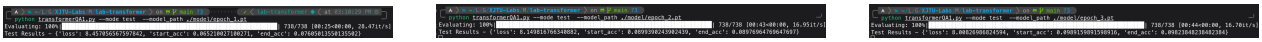

图 2: 任务一：epoch_1/2/3 模型在测试模式下的结果截图

表 1: 任务一测试指标（来自三次测试截图）

模型	Test loss	start_acc	end_acc
epoch_1.pt	8.4571	0.0652	0.0761
epoch_2.pt	8.1498	0.0899	0.0898
epoch_3.pt	8.0083	0.0989	0.0982

结果分析：

- 随 epoch 增加，测试损失从 8.4571 下降到 8.0083，start/end 准确率持续上升。
- 说明 Encoder 抽取式模型已经学习到一定的对齐关系，但绝对精度仍偏低，任务仍有较大优化空间。
- 结合训练日志，模型训练损失下降明显快于测试指标提升，存在一定泛化瓶颈。

4.2 任务二结果：Encoder-Decoder 测试损失

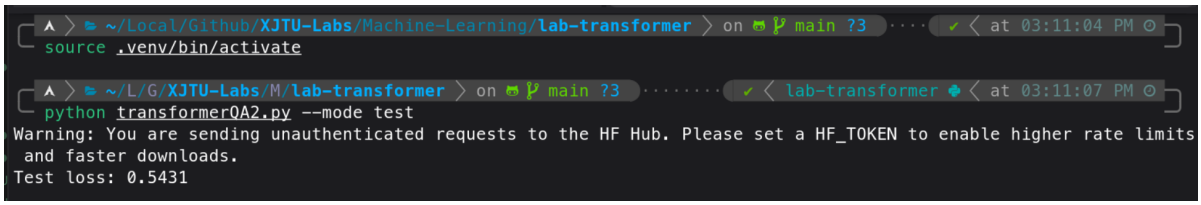

图 3: 任务二测试结果截图（`python transformerQA2.py --mode test`）

同时根据 `result/QA2_running_result.txt` 的训练日志，3 个 epoch 的指标如下：

表 2: 任务二训练/验证损失变化

Epoch	train_loss	dev_loss
1	2.9827	1.7377
2	1.2329	0.8976
3	0.6251	0.5431

结果分析：

- 任务二的损失下降趋势明显且稳定，最终 dev/test loss 约为 0.5431。

- 相比任务一，生成式模型在当前配置下给出了更低的 loss 指标。
- 但该指标仅衡量 token 级别拟合程度，后续还需引入 EM/F1 或 BLEU 等文本质量指标进行更全面评估。

5 总结

本实验完成了 SQuAD 自动问答任务中的两类 Transformer 建模：

- 完成了基于 Transformer Encoder 的抽取式问答训练与多轮测试，验证了模型随训练轮次的收敛趋势。
- 完成了基于自定义 Encoder-Decoder 的生成式问答训练与测试，获得了更低的验证损失。
- 在示例代码基础上实现了关键工程改进（AMP、并行数据加载、掩码修正、维度流修正），提高了代码可运行性与训练效率。

后续可继续从模型规模、学习率调度、答案后处理与评价指标扩展（EM/F1）等方向优化结果。

附录：实验源代码与运行命令

A.1 transformerQA1.py

```
import json
import torch
import torch.nn as nn
from torch.amp import GradScaler, autocast
from torch.utils.data import Dataset, DataLoader
from torch.optim import AdamW
import argparse
import os
from tqdm import tqdm
from transformers import BertTokenizerFast


class Config:
    def __init__(self):
        self.batch_size = 8
        self.learning_rate = 3e-5
        self.epochs = 3
```

```
        self.max_length = 384
        self.model_dir = "./model"
        self.train_path = "SQuAD-train-v2.0.json"
        self.dev_path = "SQuAD-dev-v2.0.json"
        self.device = torch.device("cuda" if torch.cuda.is_available()
            else "cpu")
        self.d_model = 768
        self.nhead = 12
        self.dim_feedforward = 3072
        self.dropout = 0.1
        self.num_layers = 7
        self.vocab_size = 30522
        self.num_workers = max(1, (os.cpu_count() or 1) - 1)
        self.pin_memory = torch.cuda.is_available()
        self.prefetch_factor = 4
        self.use_amp = torch.cuda.is_available()
        # torch.compile may be slower/noisier on smaller GPUs; keep it
            opt-in.
        self.use_compile = False


def optimize_runtime(config):
    if torch.cuda.is_available():
        torch.backends.cuda.matmul.allow_tf32 = True
        torch.backends.cudnn.allow_tf32 = True
        torch.backends.cudnn.benchmark = True
    if hasattr(torch, "set_float32_matmul_precision"):
        torch.set_float32_matmul_precision("high")


class SQuADProcessor:
    def __init__(self, config):
        self.config = config
        self.tokenizer = BertTokenizerFast.from_pretrained("bert-base-
            uncased")

    def load_data(self, path):
        with open(path, "r") as f:
            return json.load(f)["data"]

    def process(self, data):
```

```
        examples = []
        for article in data:
            for para in article["paragraphs"]:
                context = para["context"]
                for qa in para["qas"]:
                    if not qa["answers"]:
                        continue
                    example = {
                        "context": context,
                        "question": qa["question"],
                        "answer": qa["answers"][0],
                    }
                    examples.append(example)
        return examples

    def create_features(self, examples):
        input_ids, masks = [], []
        start_pos, end_pos = [], []

        for ex in examples:
            encoding = self.tokenizer(
                ex["question"],
                ex["context"],
                max_length=self.config.max_length,
                truncation=True,
                padding="max_length",
                return_offsets_mapping=True,
            )

            ans_start = ex["answer"]["answer_start"]
            ans_end = ans_start + len(ex["answer"]["text"])

            start_char = ans_start
            end_char = ans_end
            sequence_ids = encoding.sequence_ids()

            # Find token positions
            start_token, end_token = -1, -1
            for i, (idx, (s, e)) in enumerate(
                zip(sequence_ids, encoding.offset_mapping)
            ):
```

```
                if idx != 1:
                    continue  # Only look at context tokens
                if s <= start_char < e:
                    start_token = i
                if s < end_char <= e:
                    end_token = i

            if start_token != -1 and end_token != -1:
                input_ids.append(encoding["input_ids"])
                masks.append(encoding["attention_mask"])
                start_pos.append(start_token)
                end_pos.append(end_token)

        return {
            "input_ids": torch.tensor(input_ids),
            "attention_mask": torch.tensor(masks),
            "start_pos": torch.tensor(start_pos),
            "end_pos": torch.tensor(end_pos),
        }


class SQuADDataset(Dataset):
    def __init__(self, features):
        self.features = features

    def __len__(self):
        return len(self.features["input_ids"])

    def __getitem__(self, idx):
        return {k: v[idx] for k, v in self.features.items()}


class TransformerQA(nn.Module):
    def __init__(self, config):
        super().__init__()
        self.embedding = nn.Embedding(config.vocab_size, config.d_model
            )
        self.pos_encoder = nn.Parameter(torch.zeros(config.max_length,
            config.d_model))

        encoder_layer = nn.TransformerEncoderLayer(
```

```
            d_model=config.d_model,
            nhead=config.nhead,
            dim_feedforward=config.dim_feedforward,
            dropout=config.dropout,
            batch_first=True,
        )
        self.encoder = nn.TransformerEncoder(encoder_layer, config.
           num_layers)
        self.start_fc = nn.Linear(config.d_model, 1)
        self.end_fc = nn.Linear(config.d_model, 1)
        self.dropout = nn.Dropout(config.dropout)

    def forward(self, input_ids, attention_mask):
        x = self.embedding(input_ids) + self.pos_encoder[: input_ids.
           size(1)]
        x = self.dropout(x)

        output = self.encoder(x, src_key_padding_mask=~attention_mask.
           bool())

        start_logits = self.start_fc(output).squeeze(-1)
        end_logits = self.end_fc(output).squeeze(-1)
        return start_logits, end_logits


class QATrainer:
    def __init__(self, config, model, train_loader, dev_loader=None):
        self.config = config
        self.model = model.to(config.device)
        self.train_loader = train_loader
        self.dev_loader = dev_loader
        self.optimizer = AdamW(model.parameters(), lr=config.
           learning_rate)
        self.criterion = nn.CrossEntropyLoss()
        self.scaler = GradScaler("cuda", enabled=(config.use_amp and
           config.device.type == "cuda"))

        if config.use_compile:
            try:
                self.model = torch.compile(self.model)
            except Exception as e:
```

```
            print(f"torch.compile disabled: {e}")
            self.config.use_compile = False

    def train_epoch(self):
        self.model.train()
        total_loss = 0
        for batch in tqdm(self.train_loader, desc="Training"):
            input_ids = batch["input_ids"].to(self.config.device,
               non_blocking=True)
            mask = batch["attention_mask"].to(self.config.device,
               non_blocking=True)
            start = batch["start_pos"].to(self.config.device,
               non_blocking=True)
            end = batch["end_pos"].to(self.config.device, non_blocking=
               True)

            self.optimizer.zero_grad()

            with autocast("cuda", enabled=(self.config.use_amp and self
               .config.device.type == "cuda")):
                s_logits, e_logits = self.model(input_ids, mask)
                loss = self.criterion(s_logits, start) + self.criterion
                   (e_logits, end)

            self.scaler.scale(loss).backward()
            self.scaler.step(self.optimizer)
            self.scaler.update()
            total_loss += loss.item()
        return total_loss / len(self.train_loader)

    def evaluate(self):
        self.model.eval()
        total_loss = 0.0
        total_start_acc = 0.0
        total_end_acc = 0.0
        with torch.inference_mode():
            for batch in tqdm(self.dev_loader, desc="Evaluating"):
                input_ids = batch["input_ids"].to(self.config.device,
                   non_blocking=True)
                mask = batch["attention_mask"].to(self.config.device,
                   non_blocking=True)
```

```
                start = batch["start_pos"].to(self.config.device,
                    non_blocking=True)
                end = batch["end_pos"].to(self.config.device,
                    non_blocking=True)

                with autocast("cuda", enabled=(self.config.use_amp and
                    self.config.device.type == "cuda")):
                    s_logits, e_logits = self.model(input_ids, mask)
                    loss = self.criterion(s_logits, start) + self.
                        criterion(e_logits, end)
                total_loss += loss.item()

                pred_start = torch.argmax(s_logits, dim=-1)
                pred_end = torch.argmax(e_logits, dim=-1)
                total_start_acc += (pred_start == start).float().mean()
                    .item()
                total_end_acc += (pred_end == end).float().mean().item
                    ()

        metrics = {
            "loss": total_loss / len(self.dev_loader),
            "start_acc": total_start_acc / len(self.dev_loader),
            "end_acc": total_end_acc / len(self.dev_loader),
        }
        return metrics

    def save_model(self, path):
        torch.save(self.model.state_dict(), path)

    def load_model(self, path):
        self.model.load_state_dict(torch.load(path))


def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--mode", choices=["train", "test"], required=
        True)
    parser.add_argument("--model_path", type=str)
    parser.add_argument(
        "--compile",
        action="store_true",
```

```
        help="Enable torch.compile (may help on powerful GPUs, but can
            be slower on small GPUs).",
    )
    args = parser.parse_args()

    # 训练命令: python script.py --mode train
    # 测试命令: python script.py --mode test --model_path /path/to/
        model

    config = Config()
    config.use_compile = args.compile or os.environ.get("
        USE_TORCH_COMPILE", "0") == "1"
    optimize_runtime(config)
    processor = SQuADProcessor(config)

    if args.mode == "train":
        os.makedirs(config.model_dir, exist_ok=True)
        train_data = processor.load_data(config.train_path)
        train_examples = processor.process(train_data)
        train_features = processor.create_features(train_examples)
        train_dataset = SQuADDataset(train_features)
        train_loader_kwargs = {
            "batch_size": config.batch_size,
            "shuffle": True,
            "num_workers": config.num_workers,
            "pin_memory": config.pin_memory,
        }
        if config.num_workers > 0:
            train_loader_kwargs["persistent_workers"] = True
            train_loader_kwargs["prefetch_factor"] = config.
                prefetch_factor
        train_loader = DataLoader(train_dataset, **train_loader_kwargs)

        dev_data = processor.load_data(config.dev_path)
        dev_examples = processor.process(dev_data)
        dev_features = processor.create_features(dev_examples)
        dev_loader_kwargs = {
            "batch_size": config.batch_size,
            "num_workers": config.num_workers,
            "pin_memory": config.pin_memory,
        }
```

```
        if config.num_workers > 0:
            dev_loader_kwargs["persistent_workers"] = True
            dev_loader_kwargs["prefetch_factor"] = config.
               prefetch_factor
        dev_loader = DataLoader(SQuADDataset(dev_features), **
           dev_loader_kwargs)

        model = TransformerQA(config)
        trainer = QATrainer(config, model, train_loader, dev_loader)

        for epoch in range(config.epochs):
            avg_loss = trainer.train_epoch()
            print(f"Epoch {epoch+1} Loss: {avg_loss:.4f}")
            metrics = trainer.evaluate()
            print(
                f"Eval loss={metrics['loss']:.4f}, start_acc={metrics['
                   start_acc']:.4f}, end_acc={metrics['end_acc']:.4f}"
            )
            trainer.save_model(f"{config.model_dir}/epoch_{epoch+1}.pt"
               )
            if args.model_path:
                trainer.save_model(args.model_path)

    elif args.mode == "test":
        # 测试流程
        dev_data = processor.load_data(config.dev_path)
        dev_examples = processor.process(dev_data)
        dev_features = processor.create_features(dev_examples)
        dev_loader_kwargs = {
            "batch_size": config.batch_size,
            "num_workers": config.num_workers,
            "pin_memory": config.pin_memory,
        }
        if config.num_workers > 0:
            dev_loader_kwargs["persistent_workers"] = True
            dev_loader_kwargs["prefetch_factor"] = config.
               prefetch_factor
        dev_loader = DataLoader(SQuADDataset(dev_features), **
           dev_loader_kwargs)

        model = TransformerQA(config)
```

```
        trainer = QATrainer(config, model, None, dev_loader)
        trainer.load_model(args.model_path)
        metrics = trainer.evaluate()
        print(f"Test Results - {metrics}")


if __name__ == "__main__":
    main()
```

A.2 transformerQA2.py

```
    import math
import torch
import torch.nn as nn
import json
from torch.utils.data import Dataset, DataLoader
from transformers import BertTokenizer
from transformerRaw import TransformerEncoder, TransformerDecoder


class QADataset(Dataset):
    """SQuAD格式数据处理"""

    def __init__(self, file_path, tokenizer, max_length=512):
        self.data = []
        with open(file_path) as f:
            squad_data = json.load(f)

        for article in squad_data["data"]:
            for paragraph in article["paragraphs"]:
                context = paragraph["context"]
                for qa in paragraph["qas"]:
                    question = qa["question"]
                    if not qa["answers"]:
                        continue
                    answer = qa["answers"][0]["text"]

                    # 构造模型输入输出
                    input_text = f"[CLS] {question} [SEP] {context} [
                        SEP]"
```

```
                    output_text = f"[CLS] {answer} [SEP]"

                    # 编码文本
                    inputs = tokenizer(
                        input_text,
                        max_length=max_length,
                        padding="max_length",
                        truncation=True,
                        return_tensors="pt",
                    )
                    targets = tokenizer(
                        output_text,
                        max_length=max_length,
                        padding="max_length",
                        truncation=True,
                        return_tensors="pt",
                    )

                    self.data.append(
                        {
                            "input_ids": inputs["input_ids"].squeeze(0)
                                ,
                            "attention_mask": inputs["attention_mask"].
                                squeeze(0),
                            "labels": targets["input_ids"].squeeze(0),
                        }
                    )

    def __len__(self):
        return len(self.data)

    def __getitem__(self, idx):
        return self.data[idx]


class PositionalEncoding(nn.Module):
    """位置编码模块"""

    def __init__(self, d_model, max_len=512, dropout=0.1):
        super().__init__()
        self.dropout = nn.Dropout(p=dropout)
```

```
        position = torch.arange(max_len).unsqueeze(1)
        div_term = torch.exp(
            torch.arange(0, d_model, 2) * (-math.log(10000.0) / d_model
                )
        )
        pe = torch.zeros(max_len, d_model)
        pe[:, 0::2] = torch.sin(position * div_term)
        pe[:, 1::2] = torch.cos(position * div_term)
        self.register_buffer("pe", pe)

    def forward(self, x):
        x = x + self.pe[: x.size(1), :]
        return self.dropout(x)


class TransformerQA(nn.Module):
    """问答模型（含自定义编码器/解码器）"""

    def __init__(
        self,
        vocab_size,
        tokenizer,
        d_model=512,
        nhead=8,
        num_encoder_layers=6,
        num_decoder_layers=6,
        dim_feedforward=2048,
        dropout=0.1,
    ):
        super().__init__()
        self.tokenizer = tokenizer
        self.embedding = nn.Embedding(vocab_size, d_model)
        self.pos_encoder = PositionalEncoding(d_model, dropout=dropout)

        self.encoder = TransformerEncoder(
            num_encoder_layers, d_model, nhead, dim_feedforward,
                dropout
        )

        self.decoder = TransformerDecoder(
            num_decoder_layers, d_model, nhead, dim_feedforward,
```

```
            dropout
        )

        self.fc_out = nn.Linear(d_model, vocab_size)
        self.d_model = d_model

    def forward(self, src, tgt, src_mask=None, tgt_mask=None):
        # 编码器处理
        src = self.embedding(src) * math.sqrt(self.d_model)
        src = self.pos_encoder(src)
        memory = self.encoder(src, src_mask)

        # 解码器处理
        tgt = self.embedding(tgt) * math.sqrt(self.d_model)
        tgt = self.pos_encoder(tgt)
        output = self.decoder(tgt, memory, tgt_mask, src_mask)

        return self.fc_out(output)

    def generate_mask(self, src, tgt):
        """生成注意力掩码"""
        # 源序列填充掩码
        src_pad_mask = (src == 0).unsqueeze(1).unsqueeze(2)

        # 目标序列掩码 (自回归+填充)
        tgt_pad_mask = (tgt == 0).unsqueeze(1).unsqueeze(2)
        seq_len = tgt.size(1)
        tgt_sub_mask = (
            torch.triu(torch.ones(seq_len, seq_len), diagonal=1).bool()
                .to(src.device)
        )
        tgt_mask = tgt_pad_mask | tgt_sub_mask

        return src_pad_mask, tgt_mask

    # 实现自回归生成过程
    def generate_answer(self, src, max_len=50):
        if src.dim() == 1:
            src = src.unsqueeze(0)

        src_mask = (src == 0).unsqueeze(1).unsqueeze(2)
```

```
        src_emb = self.embedding(src) * math.sqrt(self.d_model)
        src_emb = self.pos_encoder(src_emb)
        memory = self.encoder(src_emb, src_mask)

        outputs = torch.LongTensor([[self.tokenizer.cls_token_id]]).to(
            src.device)

        for _ in range(max_len):
            _, tgt_mask = self.generate_mask(src, outputs)
            tgt_emb = self.embedding(outputs) * math.sqrt(self.d_model)
            tgt_emb = self.pos_encoder(tgt_emb)
            out = self.decoder(tgt_emb, memory, tgt_mask, src_mask)
            out = self.fc_out(out)
            next_token = out.argmax(-1)[:, -1:]
            outputs = torch.cat([outputs, next_token], dim=-1)

            if next_token.item() == self.tokenizer.sep_token_id:
                break

        return self.tokenizer.decode(outputs[0], skip_special_tokens=
            True)


class QATrainer:
    """训练管理类"""

    def __init__(self, model, tokenizer, device="cuda"):
        self.model = model.to(device)
        self.tokenizer = tokenizer
        self.device = device
        self.criterion = nn.CrossEntropyLoss(ignore_index=tokenizer.
            pad_token_id)

    def prepare_batch(self, batch):
        """数据预处理"""
        src = batch["input_ids"].to(self.device)  # 源序列（问题+上下
            文）
        tgt = batch["labels"].to(self.device)  # 目标序列（答案）
        # 注意：batch是否在前。

        # 构造解码器输入（去尾）和输出（去头）
```

```
        decoder_input = tgt[:, :-1]  # 移除最后一个token
        decoder_output = tgt[:, 1:]  # 移除第一个token

        # 生成注意力掩码
        src_mask, tgt_mask = self.model.generate_mask(src,
            decoder_input)
        return src, decoder_input, decoder_output, src_mask, tgt_mask

    def train_epoch(self, dataloader, optimizer):
        self.model.train()
        total_loss = 0

        for batch_idx, batch in enumerate(dataloader):
            # 数据准备
            src, decoder_in, decoder_out, src_mask, tgt_mask = self.
                prepare_batch(batch)

            optimizer.zero_grad()

            # 前向传播 (B, S) -> (B, S, V)
            # 直接使用目标序列来训练
            outputs = self.model(
                src,
                decoder_in,
                src_mask,
                tgt_mask,
            )

            # 计算损失
            loss = self.criterion(
                outputs.reshape(-1, outputs.shape[-1]), decoder_out.
                    reshape(-1)
            )

            loss.backward()
            optimizer.step()
            total_loss += loss.item()
            # 每100批次打印进度
            if batch_idx % 100 == 0:
                current_loss = total_loss / (batch_idx + 1)
                print(f"  Batch {batch_idx} | Loss: {current_loss:.4f}"
```

```
                )

        return total_loss / len(dataloader)

    def evaluate(self, dataloader):
        self.model.eval()
        total_loss = 0.0
        with torch.no_grad():
            for batch in dataloader:
                src, decoder_in, decoder_out, src_mask, tgt_mask = self
                    .prepare_batch(
                        batch
                )
                outputs = self.model(
                    src, decoder_in, src_mask, tgt_mask
                )
                loss = self.criterion(
                    outputs.reshape(-1, outputs.shape[-1]), decoder_out
                        .reshape(-1)
                )
                total_loss += loss.item()
        return total_loss / len(dataloader)


# 使用示例
if __name__ == "__main__":
    import argparse

    parser = argparse.ArgumentParser()
    parser.add_argument("--mode", choices=["train", "test"], required=
        True)
    parser.add_argument("--train_path", type=str, default="SQuAD-train-
        v2.0.json")
    parser.add_argument("--dev_path", type=str, default="SQuAD-dev-v2
        .0.json")
    parser.add_argument("--model_path", type=str, default="
        transformer_seq2seq.pt")
    parser.add_argument("--batch_size", type=int, default=8)
    parser.add_argument("--epochs", type=int, default=3)
    parser.add_argument("--max_length", type=int, default=256)
    parser.add_argument("--lr", type=float, default=3e-5)
```

```
    args = parser.parse_args()

    tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu
        ")

    if args.mode == "train":
        train_dataset = QADataset(
            args.train_path, tokenizer, max_length=args.max_length
        )
        dev_dataset = QADataset(args.dev_path, tokenizer, max_length=
            args.max_length)
        train_loader = DataLoader(
            train_dataset, batch_size=args.batch_size, shuffle=True
        )
        dev_loader = DataLoader(dev_dataset, batch_size=args.batch_size
            )

        model = TransformerQA(vocab_size=tokenizer.vocab_size,
            tokenizer=tokenizer)
        trainer = QATrainer(model, tokenizer, device=device)
        optimizer = torch.optim.Adam(model.parameters(), lr=args.lr)

        for epoch in range(args.epochs):
            train_loss = trainer.train_epoch(train_loader, optimizer)
            eval_loss = trainer.evaluate(dev_loader)
            print(
                f"Epoch {epoch+1} | train_loss={train_loss:.4f} |
                    dev_loss={eval_loss:.4f}"
            )
        torch.save(model.state_dict(), args.model_path)
        print(f"Saved model to {args.model_path}")
    else:
        dev_dataset = QADataset(args.dev_path, tokenizer, max_length=
            args.max_length)
        dev_loader = DataLoader(dev_dataset, batch_size=args.batch_size
            )
        model = TransformerQA(vocab_size=tokenizer.vocab_size,
            tokenizer=tokenizer)
        model.load_state_dict(torch.load(args.model_path, map_location=
            device))
```

```
        model.to(device)
        trainer = QATrainer(model, tokenizer, device=device)
        test_loss = trainer.evaluate(dev_loader)
        print(f"Test loss: {test_loss:.4f}")
```

A.2 运行命令

```
# 任务一训练
python transformerQA1.py --mode train

# 任务一测试（分别测试三个 epoch）
python transformerQA1.py --mode test --model_path ./model/epoch_1.pt
python transformerQA1.py --mode test --model_path ./model/epoch_2.pt
python transformerQA1.py --mode test --model_path ./model/epoch_3.pt

# 任务二训练与测试
python transformerQA2.py --mode train
python transformerQA2.py --mode test
```